"Politehnica" University of Timisoara

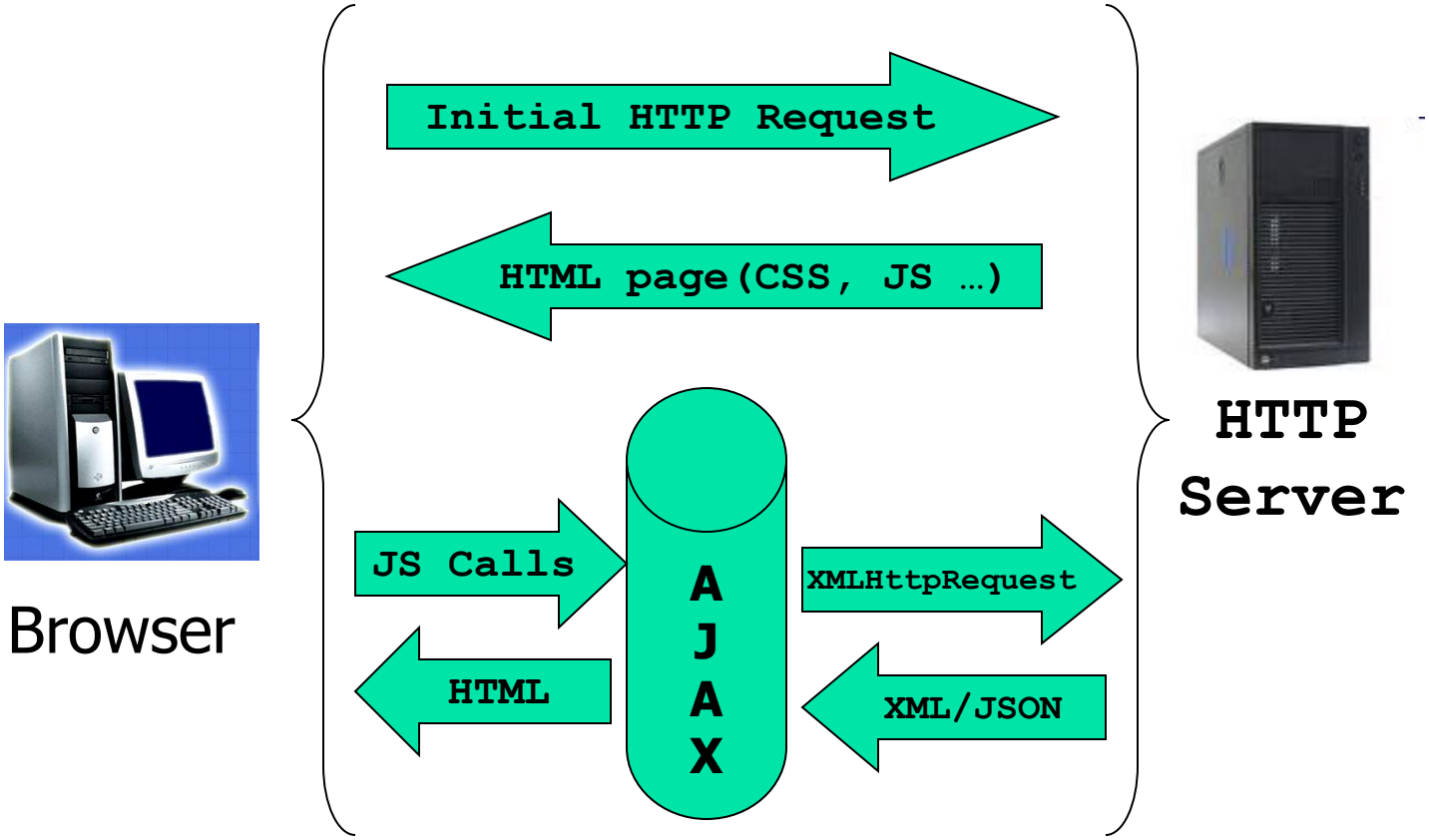
Web Programming

C4. Asynchronous JavaScript

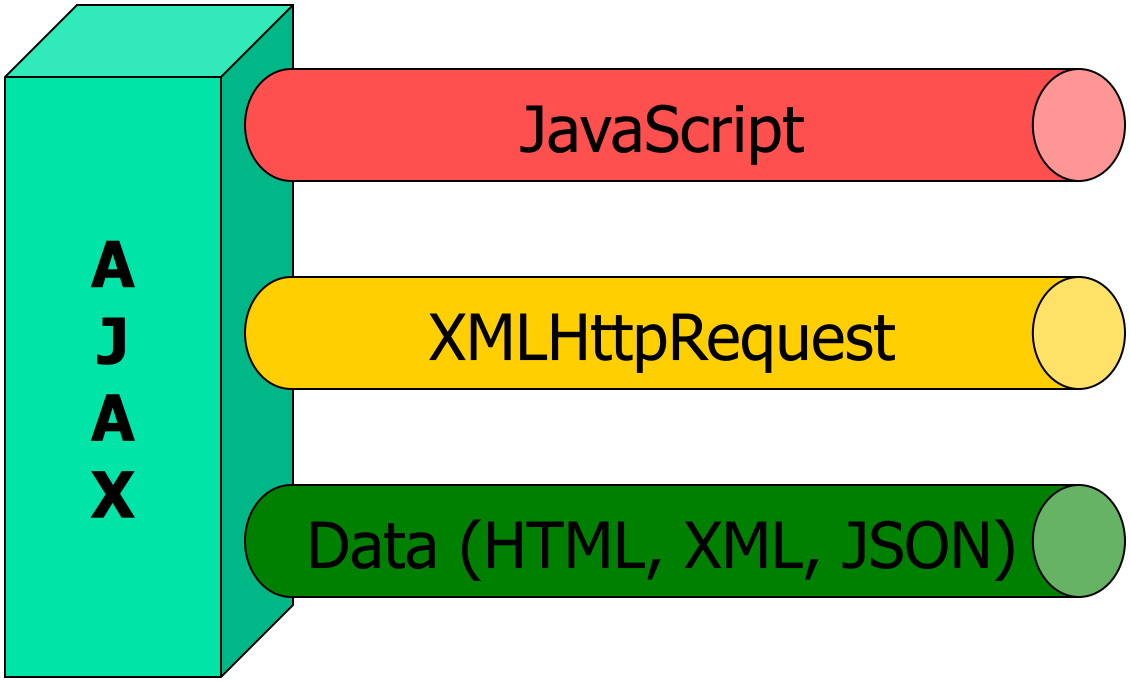
Ajax

- **AJAX** - **A**synchronous **JA**vaScript and **X**ML
- A new approach to implement Web applications
- Create "true" interactive web applications
- Based on JavaScript *XMLHttpRequest* object
- For security reasons, modern browsers do not allow access across domains
  - The data (XML/JSON) files must be located on the page originating server

Ajax in Action



Fundaments of Ajax



Browser Support

- Most browsers have support for XMLHttpRequest

```
function ajaxFunction()
{
    var xmlhttp;
    if (window.XMLHttpRequest) {
        // for IE7+, Firefox, Opera, Safari, Chrome
        xmlhttp=new XMLHttpRequest();
    }
    else if (window.ActiveXObject)
      {
         // for IE6, IE5
         xmlhttp=new ActiveXObject
                ("Microsoft.XMLHTTP");
      }
      else
        {
          alert("Your browser does not support XMLHTTP!");
        }
}
```

XMLHttpRequest (I)

- XMLHttpRequest object properties:
  - **onreadystatechange**. Allows implementation of a function to process data received from the server. This function will be called automatically by the browser when associated xmlHttp event occurs.

```
xmlhttp.onreadystatechange = function()
{
   // data processing
}
```

XMLHttpRequest (II)

- XMLHttpRequest object properties:
  - **readyState**. This property keeps status code that is the answer to the last call made. Possible values are:
    0 – application was not initialized;
    1 – application is ready;
    2 – the request has been sent;
    3 – the request is in process;
    4 – the request is complete.
  - **status**. Returns the request status number. Possible values are:
    200 – OK;
    403 – Forbidden;
    404 – Not Found.

XMLHttpRequest (III)

- **responseText / responseXML.** Provides data from the server response. They are usually wrapped in plain text/HTML, JSON or XML.

```
xmlhttp.onreadystatechange = function()
{
  if(xmlhttp.readyState==4 and xmlhttp.status==200)
  {
    document.dispPanel.value = xmlhttp.responseText;
  }
}
```

XMLHttpRequest (IV)

- **responseXML**

```
xmlDoc=xmlhttp.responseXML;
var txt="";

x=xmlDoc.getElementsByTagName("ITEM");
for (i=0;i<x.length;i++)
{
  txt=txt + x[i].childNodes[0].nodeValue + "<br />";
}
document.getElementById("divPanel").innerHTML=txt;
```

AJAX Request To a Server

- open(*method*, *url*, *async*)
  - *method* - the type of request (GET/POST)
  - *url* – the target file URL
  - *async* - true (asynchronous/default) or false (synchronous)

```
xhttp.open("GET", "computeData.php", true);
xhttp.send();
```

AJAX GET or SET

- GET is simpler and faster than POST
  - *GET can be used to:*
    - *Request some data from the server*
    - *Avoid cached page (e.g., when update a database on the server)*

```
xhttp.open("GET","compute.php?q=" + Math.random(),true);
xhttp.send();
```

  - *POST can be used to:*
    - *Send a large amount of data to be stored on the server (POST has no size limitations as GET)*
    - *Send some sensitive user data - POST is more robust and secure*

AJAX responseText Example

```
<body>
   <div id="demo">
      <h2>The XMLHttpRequest Object</h2>
      <button type="button" onclick="loadDoc()">Change Content</button>
   </div>
   <script>
      function loadDoc() {
            const xhttp = new XMLHttpRequest();
            xhttp.onload = function() {
                 document.getElementById("demo").innerHTML = this.responseText;
            }
            xhttp.open("GET", "ajax_info.txt");
            xhttp.send();
       }
    </script>
</body>
```

Ref: https://www.w3schools.com/js/

AJAX responseXML Example

```
<button type="button" onclick="loadDoc()">Get my CD collection</button>
<br/>  <table id="demo"></table>
<script>
    function loadDoc() {
        const xhttp = new XMLHttpRequest();
        xhttp.onload = function() {  myFunction(this); }
        xhttp.open("GET", "cd_catalog.xml");
        xhttp.send();
    }
    function myFunction(xml) {
        const x = xml.responseXML.getElementsByTagName("CD");
        let table="<tr><th>Artist</th><th>Title</th></tr>";
        for (let i = 0; i <x.length; i++) {
        table += "<tr><td>"+x[i].getElementsByTagName("ARTIST")[0].childNodes[0].nodeValue+
        "</td><td>"+x[i].getElementsByTagName("TITLE")[0].childNodes[0].nodeValue+"</td></tr>";
        }
    document.getElementById("demo").innerHTML = table;
}
</script>
```

Ref: https://www.w3schools.com/js/

```
<?xml version="1.0" encoding="UTF-8"?>
<CATALOG>
  <CD>
    <TITLE>Pavarotti Gala Concert</TITLE>
    <ARTIST>Luciano Pavarotti</ARTIST>
    <COUNTRY>UK</COUNTRY>
    <COMPANY>DECCA</COMPANY>
    <PRICE>9.90</PRICE>
    <YEAR>1991</YEAR>
  </CD>
  <CD>
    <TITLE>Picture book</TITLE>
    <ARTIST>Simply Red</ARTIST>
    <COUNTRY>EU</COUNTRY>
    <COMPANY>Elektra</COMPANY>
    <PRICE>7.20</PRICE>
    <YEAR>1985</YEAR>
  </CD>
</CATALOG>
```

Ajax Resources

- Ajax Tutorials

  https://www.w3schools.com/js/js_ajax_intro.asp

  http://www.tizag.com/ajaxTutorial/

  http://www.tutorialspoint.com/ajax/

Promise API

- Can be used to handle asynchronous JavaScript calls
- A Promise is an Object representing the eventual completion or failure of an asynchronous operation
  - Easy to write code implementing multiple asynchronous operations, where callbacks lead to high complexity
- It has a state (pending, fulfilled, rejected) and, if fulfilled, it returns a result

Types of Promises (I)

- Promise.all
  - Takes an iterable of promises as input and returns a single Promise
  - Waits for all of them to finish
  - If any promise is rejected, it throws an error and stops
  - Get all data(if all succeed) or error (if anyone is rejected)

Types of Promises (II)

- Promise.allSatteled
  - Get results from all promises, even if a few/all promises are rejected
- Promise.race
  - The first finisher will be returned, regardless of whether it succeeds or is rejected
- Promise.any
  - Similar to race, but will wait to get any first success result

Promise.all

```
const p1 = new Promise((success, reject) => {
    setTimeout(() => {
        success('P1 Success.');
    }, 3000)
})

const p2 = new Promise((success, reject) => {
    setTimeout(() => {
        success('P2 Success.');
    }, 1000)
})

const p3 = new Promise((success, reject) => {
    setTimeout(() => {
        reject('P3 Rejected.');
    }, 2000)
})


const allPromise = Promise.all([p1, p2, p3]);

allPromise
.then(res => console.log(JSON.stringify(res, null, 2)))
.catch(err => console.error(err))
```

```
// Will get result after 1 second
// P2 Rejected.
```

Promise.allSettled

```
const p1 = new Promise((success, reject) => {
    setTimeout(() => {
        success('P1 Success.');
    }, 3000)
})

const p2 = new Promise((success, reject) => {
    setTimeout(() => {
        success('P2 Success.');
    }, 1000)
})

const p3 = new Promise((success, reject) => {
    setTimeout(() => {
        reject('P3 Rejected.');
    }, 2000)
})

const allPromise = Promise.allSettled([p1, p2, p3]);

allPromise.then(res => console.log(JSON.stringify(res, null, 2)))
```

```
// Output

/*
[
  {
    "status": "fulfilled",
    "value": "P1 Success."
  },
  {
    "status": "fulfilled",
    "value": "P2 Success."
  },
  {
    "status": "rejected",
    "reason": "P3 Rejected."
  }
]
*/
```

Promise.race

```
const p1 = new Promise((success, reject) => {
    setTimeout(() => {
        success('P1 Success.');
    }, 3000)
})

const p2 = new Promise((success, reject) => {
    setTimeout(() => {
        reject('P2 Rejected.');
    }, 1000)
})

const p3 = new Promise((success, reject) => {
    setTimeout(() => {
        success('P3 Success.');
    }, 2000)
})


const allPromise = Promise.race([p1, p2, p3]);

allPromise
.then(res => console.log(res))
.catch(err => console.error(err))
```

```
// Will get result after 1 second
// P2 Rejected.
```

Promise.any

```
const p1 = new Promise((success, reject) => {
    setTimeout(() => {
        success('P1 Success.');
    }, 3000)
})

const p2 = new Promise((success, reject) => {
    setTimeout(() => {
        reject('P2 Rejected.');
    }, 1000)
})
const p3 = new Promise((success, reject) => {
    setTimeout(() => {
        success('P3 Success.');
    }, 2000)
})

const allPromise = Promise.any([p1, p2, p3]);
allPromise
.then(res => console.log(res))
.catch(err => console.error(err))
```

```
// Will get result after 2second from p3
// P3 Succeed.
```

JavaScript Fetch API

- Alternative to AJAX
- Used for fetching a resource from a Web server
- Built on Promise API
- The fetch() method returns a Promise that resolves to a Response object
- No need for XMLHttpRequest object

JavaScript Fetch Example

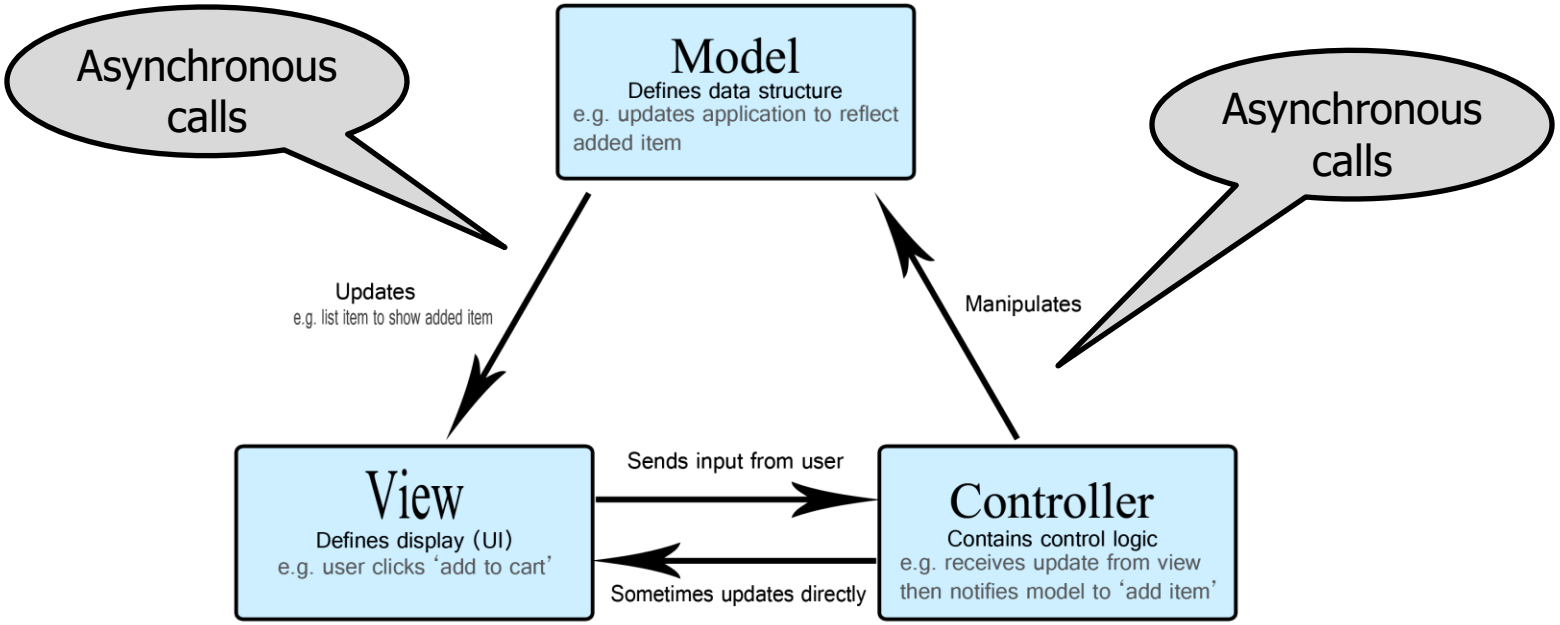
```
<body>
   <div id= "demo"> <p>Fetch a file to change this text.</p>
   <button type="button" onClick="getText('fetch_info.txt')”>
                                                    Change content</button>
   </div>
   <script>
        async function getText(file) {
               let myObject = await fetch(file);
               let myText = await myObject.text();
               document.getElementById("demo").innerHTML = myText;
        }
    </script>
</body>
```

Compact Fetch Example

```
<body>
   <div id= "demo"> <p>Fetch a file to change this text.</p>
   <button type="button" onClick="getText('fetch_info.txt')”>
                                                    Change content</button>
   </div>
   <script>
        function getText(file) {
           fetch(file)
              .then(x => x.text())
              .then(y => document.getElementById("demo").innerHTML = y);
        }
    </script>
</body>
```

MVC model for Ajax apps

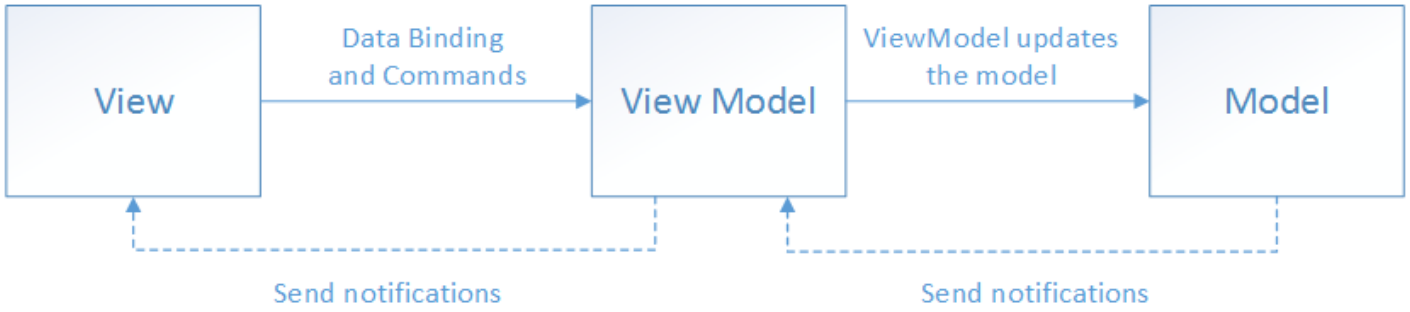
- Model View Controller – the most use model for Web apps



* Ref: https://developer.mozilla.org/

MVVM model

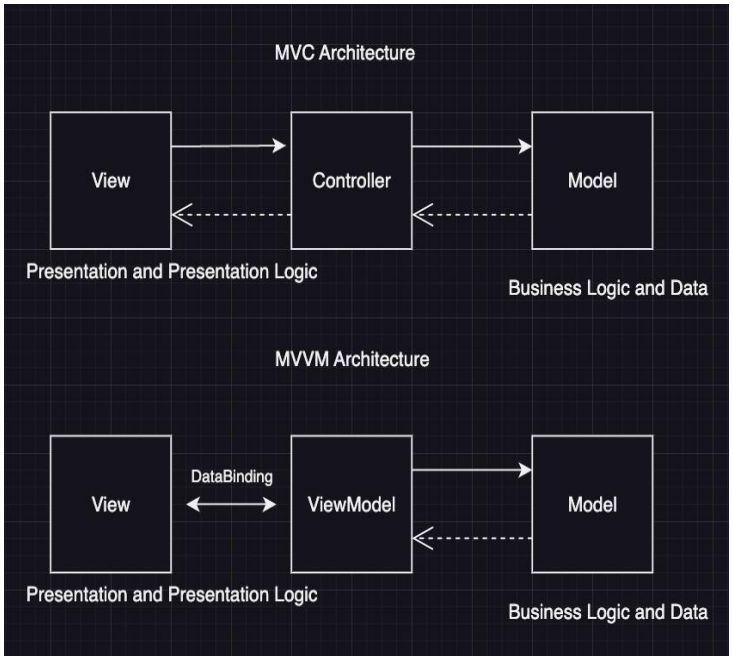
- Model View ViewModel – helps cleanly separate an application's business and presentation logic from its user interface (UI)



* Ref: https://learn.microsoft.com/

MVC / MVVM structure

- Both provide structure and separation of concerns
- In MVC
  - The Controller updates View from Model data (manual data synchronization)
- In MVVM
  - The ViewModel component acts as an intermediary between the View and the Model, exposing data and commands for data binding
  - Data binding allows automatic synchronization of data between View and ViewModel



* Ref: https://brightmarbles.io/

MVC vs. MVVM

- MVC
  - Controller receives user input, processes it, and updates the Model
  - Linear flow: User -> Controller -> Model -> View
  - Can be more complex to unit test due to the coupling between the Controller and the View
  - Best for applications with a simpler, more direct UI flow, like traditional web applications
- MVVM
  - View sends commands to the ViewModel. The ViewModel interacts with the Model. Data binding automates the UI refresh
  - Dynamix flow: User interacts with View, which communicates with ViewModel, which then interacts with the Model
  - Easier to unit test because the ViewModel can be tested independently of the View
  - Best for UI intensive apps (e.g., single-page applications – SPAs)

Popular Libraries using Ajax

- General JS programming
  - Prototype
- Other programming
  - GWT (Java)
- DOM manipulation
  - jQuery
- Rich GUIs
  - Ext-JS, Dojo, jQuery UI
- Familiarity to JS developers
  - Lowest: GWT
- Traditional Ajax support
  - Django-dajax
- Server communication
  - GWT
  - DWR, JSON-RPC
- Usage in industry
  - jQuery
  - Ext-JS, Dojo, YUI
  - Prototype, Scriptaculous, GWT, Django-dajax

Ajax based Frameworks

- **Dojo Toolkit /Dojo (http://dojotoolkit.org/ / https://dojo.io/)**
  - Is mainly a dynamic UI toolkit. It includes implementations of GUI elements, AJAX-style communication with the server, and visual effects. Dojo's widget system Dijit provides a variety of commonly used form widgets.
- **DHTMLX (http://dhtmlx.com/)**
  - Includes the most widely-used types of UI components: grid, treegrid, treeview, combobox, tabbar, menu, windows, calendar, layout, and more.
- **AngularJS (https://angularjs.org/)**
  - Is a front-end web application framework maintained by Google. It support model–view–controller (MVC) and model–view–viewmodel (MVVM) architectures.
- **React (https://reactjs.org/)**
  - Is a UI Framework used to efficiently update and render just the right components when application data changes. Fast learning curve. Supports Flux architecture. It is one of Facebook's first open-source project.

jQuery Library

- "**jQuery** is a fast, small, and feature-rich JavaScript library.
  It makes things like HTML document traversal and manipulation, event handling, animation, and Ajax much simpler with an easy-to-use API that works across a multitude of browsers."
  (http://jquery.com/)
- Alleviates the pains of cross browser development
- jQuery Examples:
  (https://webdesignerwall.com/tutorials/jquery-tutorials-for-designers/)

jQuery Features

- Selectors
  - No more getElementById()
- Wrapped Set Operations
- Events
- Plugin Extensibility
- jQueryUI Widgets
- Themes via ThemeRoller

jQuery Selectors

- Powerful Selector Engine (Sizzle)
  - $(".myElement") // by css Class
  - $("#nameTextBox") // by Id
  - $("ul") // by tag name
  - $("ul li") // element descendent
  - $("ul").find("li") // more element descendent
  - $("ul> li") // element child (direct descendent)
  - $("div":has(p)) // only containing <p>
  - $("input:checkbox") // filter
  - $("a[href=#Overview]") // attributes
  - $("a[href$=.aspx]") // attributes ends with a string

jQuery Selectors Example

```
<head>
   <script
   src = "http://code.jquery.com/jquery-1.9.1.js">
   </script>
</head>
<body>
   <p><span>Hello</span>, how are you?</p>
   <p>Me? I'm <span>good</span>.</p>
   <script>
   $("p").find("span").css("color","red");
   </script>
</body>
```

Hello, how are you?
Me? I'm good.

jQuery Wrapper Set Operations

- jQuery Wrapped Set Object
  - Every selector returns a Set (an object that acts as an array)
- jQuery Wrapped Set Operations
  - $(".myTable tr:odd").css("background", "gray");
- jQuery Wrapped Set Operation Chaining
  - $(".myDiv").css("background","gray").addClass("dottedBorder");
- Common Operations
  - $().html().text().val().attr().append().empty()
  - $().css().addClass() .removeClass().hasClass() .offset().height().width().scrollTop().scrollLeft() .show().hide()
  - $().find().is().has().not().filter().parent() .closest().next()
  - $().live().bind().click().dblclick().hover().toggle().blur() .keydown().keyup().resize() .mouseover().mousedown()

jQuery Callbacks

```
$(".myLink").click(function(e) {
        alert(this.href); // this is usually the context,
                          // here it's the link
});
```

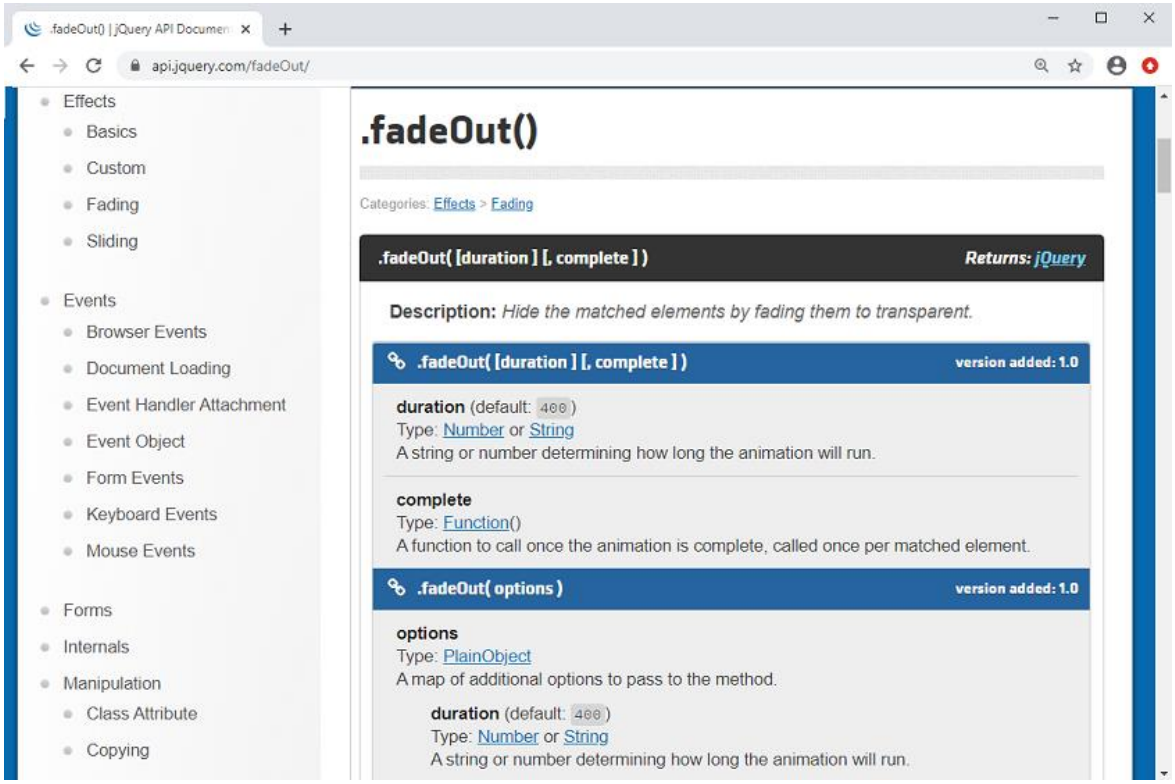
```
$(".myLink").click(myLinkClick);  // don't have to be anonymous/inline

function myLinkClick() {
        alert(this.href);
}
```

```
$(".myLink").toggle(function(e) {
        $(this).addClass('redLink'); // sometimes theirs two callbacks
}, function(e) {
        $(this).removeClass('redLink');
});
```

```
// AJAX - use GET to read google results
$.get("http://www.google.com/search", {q : "jQuery"}, function(html) {
        alert(this); // the jquery ajax object
        alert(html); // the google html results
});
```

jQuery Documentation

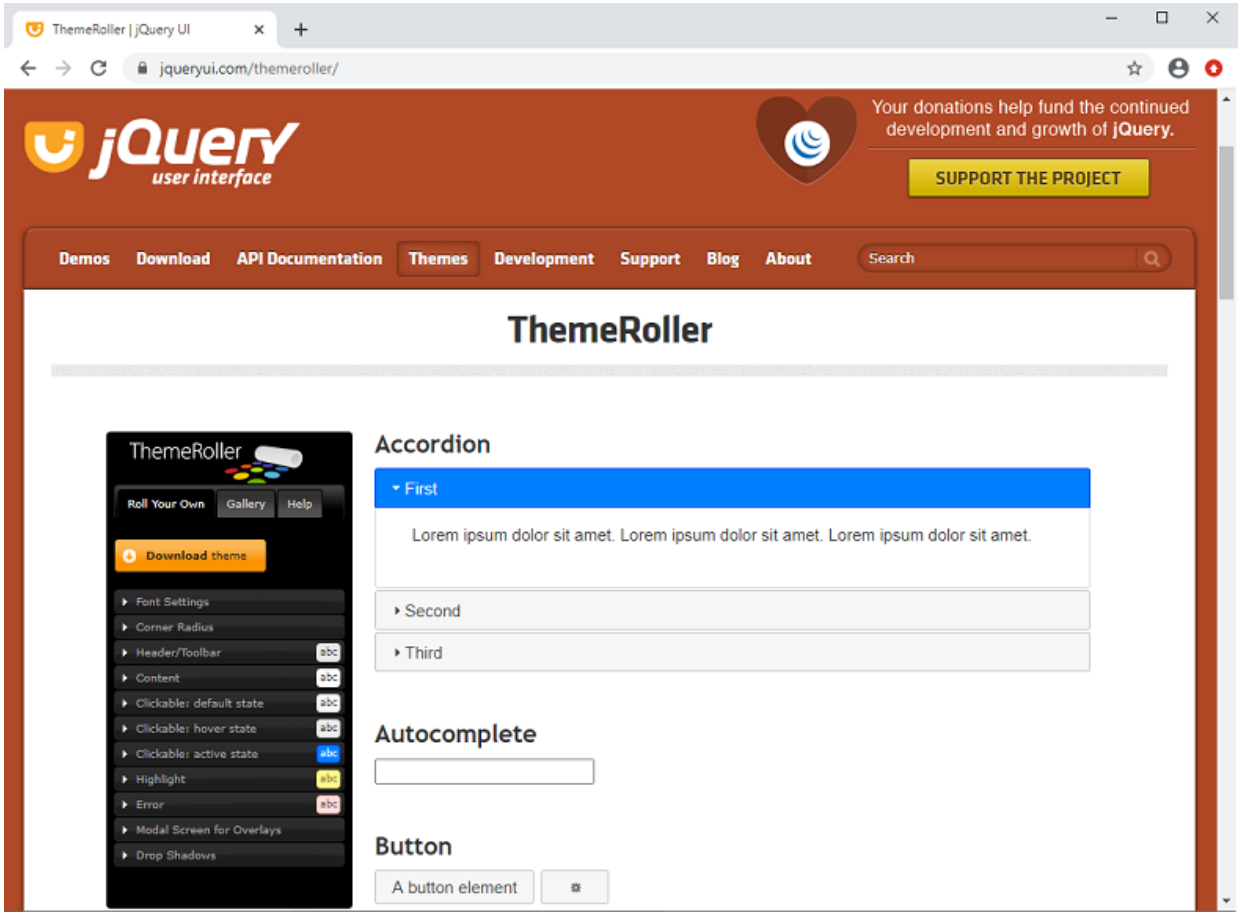
- https://api.jquery.com/



jQuery UI

- jQuery UI - built on top of the jQuery
  - User interface interactions
  - Effects (Slide, Scale, Size, Pulsate, Bounce, Highlight, Shake)
  - Widgets (Accordion, Datepicker, Dialog, Progressbar, Slider, Tabs)
  - Themes
- Reference: http://jqueryui.com/

jQuery UI – ThemeRoller



jQuery Mobile

- Mobile market growth:
  - Smartphones and tablets are everywhere
  - Data transmission rates grows (HSDPA+ & 4G) - easy/fast access to information
  - Many platforms - iOS,Android,Windows Mobile,WebOS, Symbian
  - Various browsers - Safari, Chrome, Opera, Firefox
- Heterogeneous platforms:
  - Need for a unifying technology
  - Classical solutions are not optimal
- Support for touchscreens/rotation
  - Touch input
  - Orientation events

jQuery Mobile Features

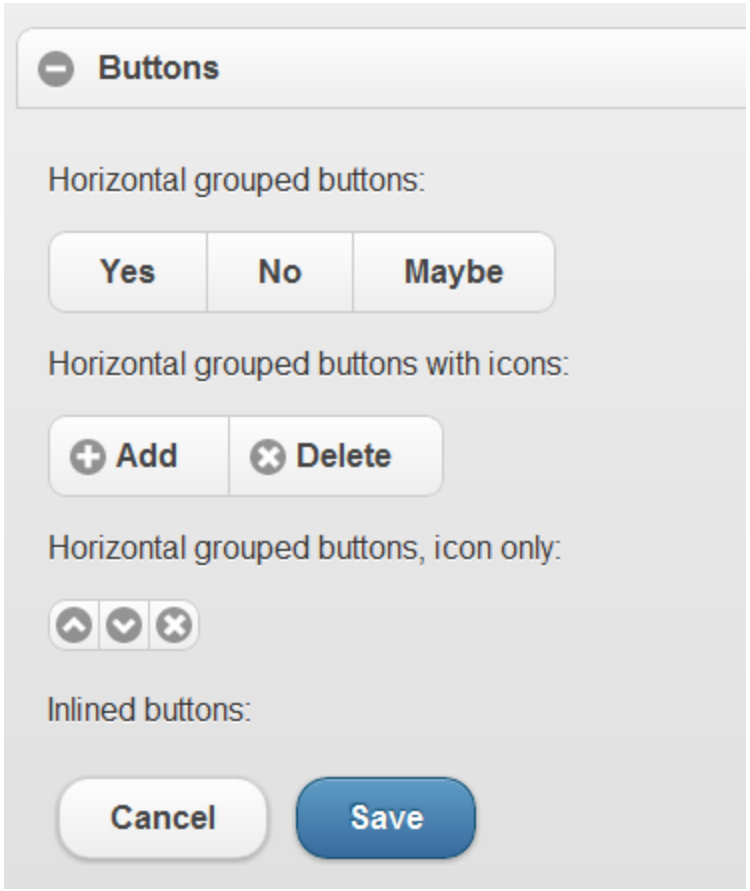
- HTML5 "data" - attribute :
  - used for defining behavior and creating elements
- Structure :
  - Consists of an element (usually a tag) with a "data-role" attribute set to "page"
  - Has elements with roles of header, content, and footer (all optional)
- Single/Multi pages :
  - Optimization for using either single pages or multiple internal pages
  - Internal pages are stacked together with single pages for easy access
  - Advantage - page links will work with standard HTML requests
- Page Linking :
  - By default, use Ajax when not available normal HTTP request
  - Default behavior can be overridden
- Page Transitions :
  - Several predefined CSS transitions are available
  - Transitions can be applied to any object or page change event

jQuery Mobile Example

```
<!DOCTYPE html>
<html>
    <head>
    <title>Page Title</title>
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <link rel="stylesheet" href="http://code.jquery.com/mobile/1.0rc2/jquery.mobile-1.0rc2.min.css" />
    <script type="text/javascript" src="http://code.jquery.com/jquery-1.6.4.min.js"></script>
    <script type="text/javascript" src="http://code.jquery.com/mobile/1.0rc2/jquery.mobile-1.0rc2.min.js"></script>
</head>
<body>
<div data-role="page">
    <div data-role="header">
        <h1>Page Title</h1>
    </div><!-- /header -->

    <div data-role="content">
        <p>Page content goes here.</p>
    </div><!-- /content -->

    <div data-role="footer">
        <h4>Page Footer</h4>
    </div><!-- /footer -->
</div><!-- /page -->
</body>
</html>
```

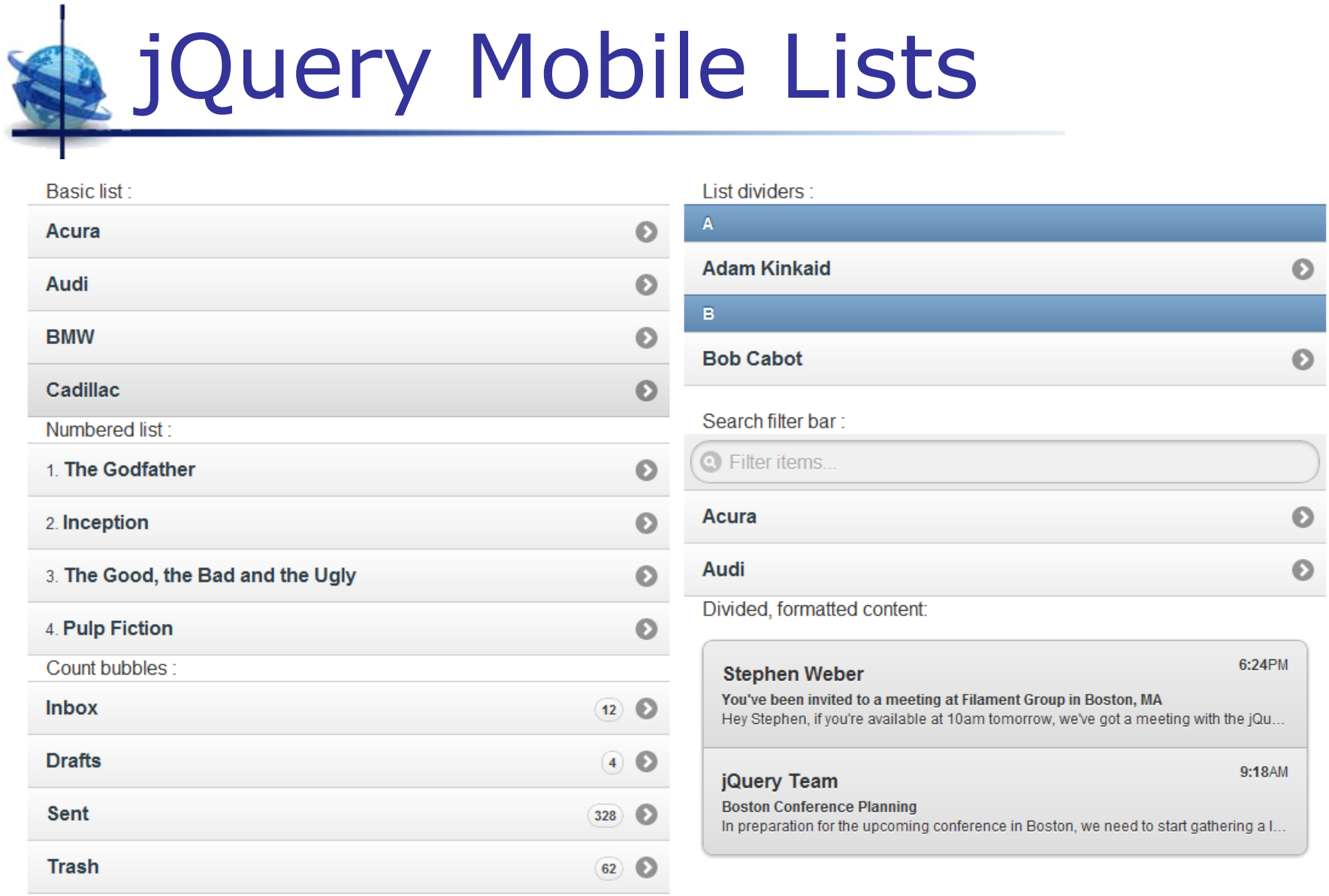
jQuery Mobile Buttons

- A lot of predefined button types
- Easy to integrate (data-role="button")
- Touch optimized

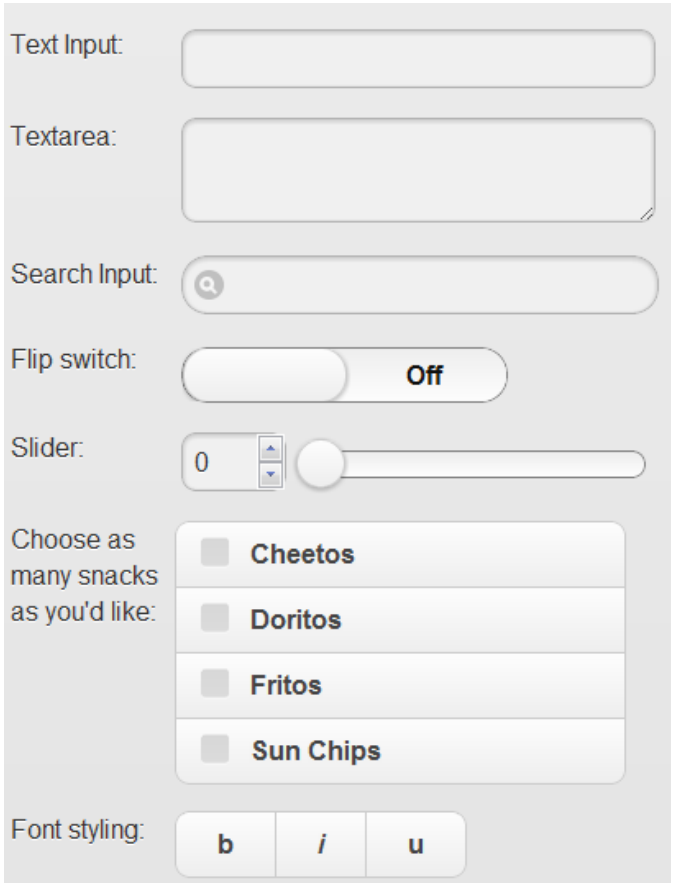


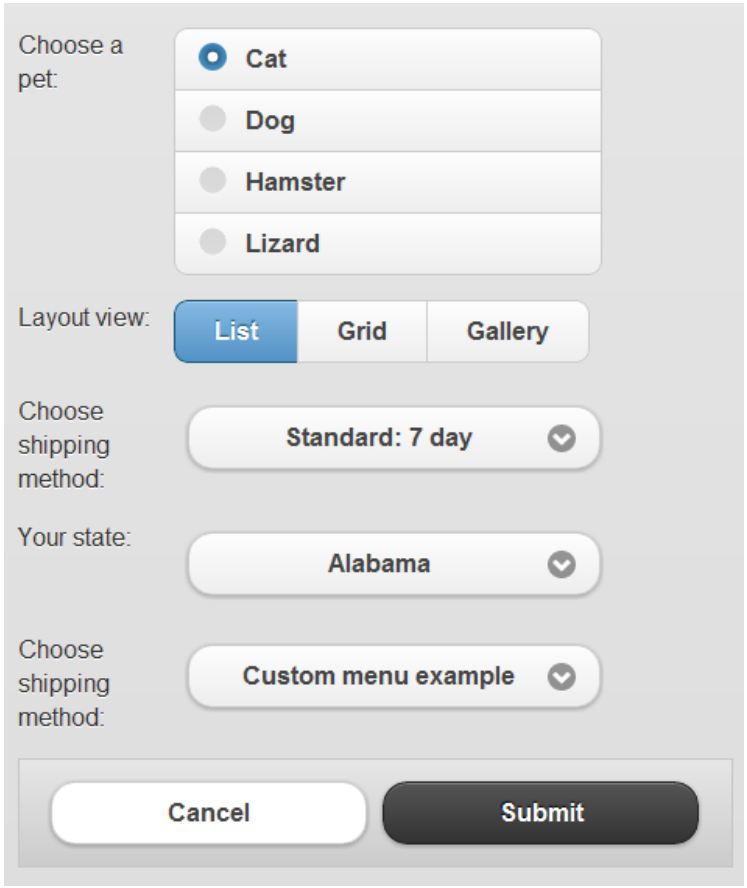
* Ref: http://jquerymobile.com

jQuery Mobile Lists



jQuery Mobile Forms





jQuery Mobile Events

- Touch input handling:
  - tap, taphold, swipe, swipeleft, swiperight
- Virtual mouse events:
  - vmouseover, vmouseup, vmousedown, vmousemove, vclick, vmousecancel
- Orientation change event:
  - orientationchange – triggers when device is turned horizontally or vertically

jQuery Mobile API

- Configuration defaults
  - via the $.mobile object
  - used for setting the default page and dialog transitions, error messages, usage of Ajax etc.
- Utilities and Methods (ex.):
  - $.mobile.changePage (method)
  - $.mobile.loadPage (method)
  - $.mobile.showPageLoadingMsg ()
  - $.mobile.fixedToolbars.show (method)
- Can override default behavior
- Reference: https://demos.jquerymobile.com/